AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

i-CREDIT HOURS ENGINEERING PROGRAMS

Loan Prediction Project – Phase 1: Data Preprocessing

Submitted to: Dr. Mahmoud Mounir & Eng. Esraa Karam

Submitted by Team 20:

1. AbdelRahman Adel 23P0144
2. Jana hussien 2200090
3. Omar Ahmed 23P0100
4. Mariam Ibrahim 23P0123
5. Hla Waleed 23P0210
6. Malak Khaled 2300447
7. Kenzy Tarek 2301141

Contents

1. Introduction 3
2. Loading and Inspecting the Data 3
3. Dropping Unnecessary Columns 3
4. Exploring Data Types and Missing Values 4
5. Understanding Relationships Between Columns 4
6. Handling Missing Values 6
7. Imputing Missing LoanAmount Using Regression 7
8. Imputing Remaining Columns 8
9. Feature Engineering 9
10. Outlier Detection and Treatment 9
11. Encoding Categorical Variables 11
12. Feature Scaling 12
13. Summary of Preprocessing Steps 13

1. Introduction

In this phase, we focused on cleaning and preparing the Loan Prediction dataset for machine learning. The goal was to handle missing values, encode categorical variables, remove outliers, and scale numerical features to ensure data consistency and model readiness.

2. Loading and Inspecting the Data

We began by importing essential Python libraries such as pandas, numpy, matplotlib, seaborn, and preprocessing tools from sklearn.
The dataset was then loaded using pd.read_csv() and displayed using df.head() to view the first few rows.

From the initial inspection:

- The dataset contained **614 rows** and **13 columns**.
- Data types included 8 categorical (object), 4 numerical (float64), and 1 integer (int64).
- The target variable was **Loan_Status**.

3. Dropping Unnecessary Columns

The **Loan_ID** column was dropped since it was only an identifier and carried no predictive value.

```
df = df.drop('Loan_ID', axis=1)
```

4. Exploring Data Types and Missing Values

We checked column types, unique values, and null counts using df.info(), df.nunique(), and df.isnull().sum().
The results showed missing values in several columns:

Column	Missing Count
Credit_History	50
Self_Employed	32
LoanAmount	22
Dependents	15
Loan_Amount_Term	14
Gender	13
Married	3

5. Understanding Relationships Between Columns

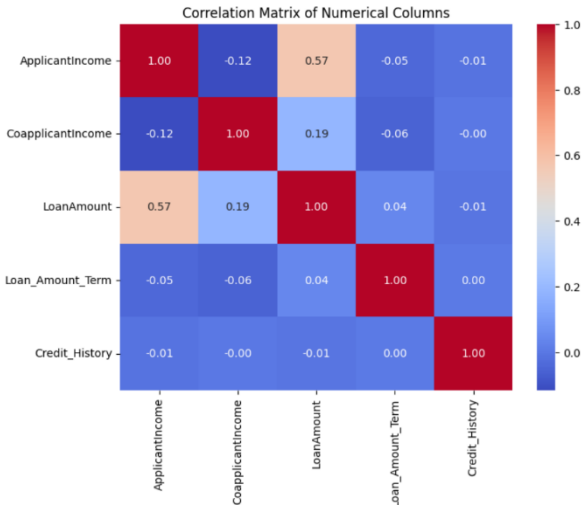
To better understand relationships between features, we visualized:

- A **correlation matrix** for numerical columns.
- Several **count plots** (Education vs. Married, Education vs. Dependents, Married vs. Dependents, Education vs. Self_Employed).

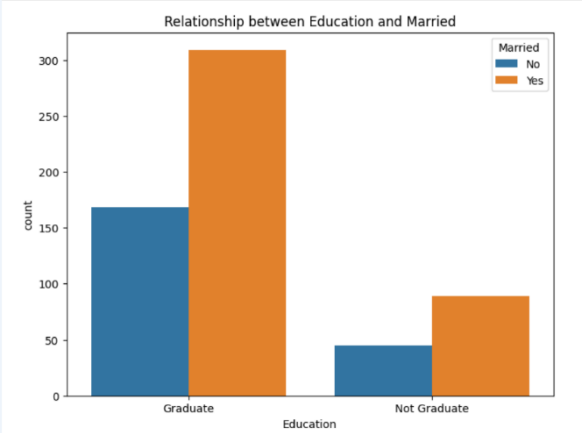
Key Insights:

- Graduates were more likely to be married than Non-Graduates.
- Married individuals usually had more dependents.
- ApplicantIncome moderately correlated (≈0.57) with LoanAmount.
- Self_Employment showed no strong correlation with Education.

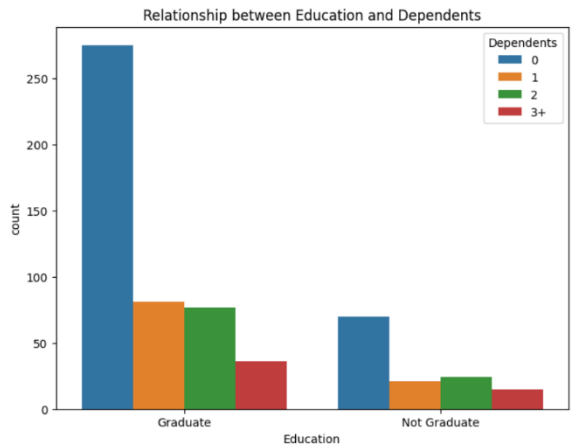
These insights guided later imputation decisions.



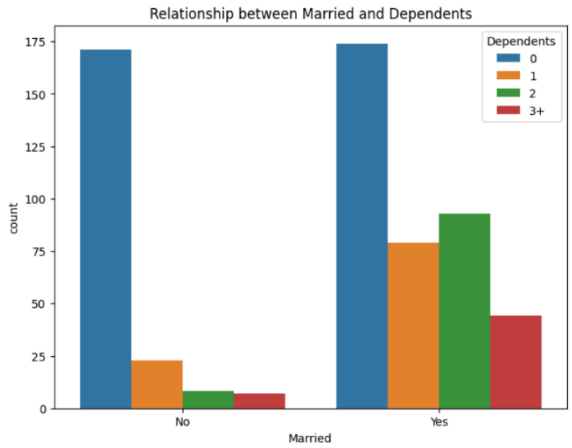
Correlation Matrix of Numerical Columns



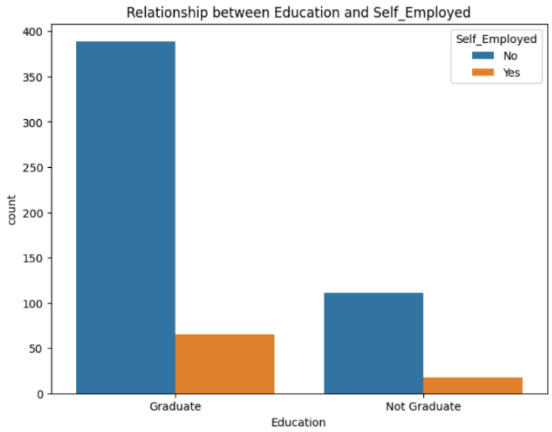
Relationship between Education and Married.



Relationship between Education and Dependents.



Relationship between Married and Dependents.



Relationship between Education and Self_Employed.

6. Handling Missing Values

a) Gender

We filled missing values in **Gender** with the most frequent category (mode):

```
df['Gender'].fillna(mode_gender, inplace=True)
```

b) Married

Missing **Married** values were filled logically based on **Education**:

- Graduates → "Yes"
- Non-Graduates → "No"

c) Dependents

Missing **Dependents** were filled using the mode within each **Married** group:

```
df['Dependents'] = df.groupby('Married')['Dependents'].transform(lambda x: x.fillna(x.mode()[0]))
```

d) Self_Employed

Filled with mode (most frequent value):

```
mode_self_employed = df['Self_Employed'].mode()[0]
df['Self_Employed'].fillna(mode_self_employed, inplace=True)
```

7. Imputing Missing LoanAmount Using Regression

Step 1: Splitting the Data

We separated the dataset into two parts:

- Rows with missing LoanAmount
- Rows with existing LoanAmount

```
df_missing_loanamount = df[df['LoanAmount'].isnull()]
df_not_missing_loanamount = df[df['LoanAmount'].notnull()]
```

Step 2: Feature Selection

Based on correlation and domain understanding, the following features were used as predictors:

['ApplicantIncome', 'CoapplicantIncome', 'Loan_Amount_Term', 'Credit_History',

'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area']

Step 3: One-Hot Encoding

Categorical features were encoded using OneHotEncoder to make them suitable for modeling.

Step 4: Model Training

We trained a **RandomForestRegressor** model on non-missing rows to predict LoanAmount.

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

Step 5: Predicting Missing Values

The trained model was used to predict missing LoanAmount values, which were then inserted back into the main DataFrame.

- ✓ After verification, **no missing values remained in LoanAmount.**

8. Imputing Remaining Columns

- **Loan_Amount_Term** and **Credit_History** were filled with their respective **mode** values.

After these steps, the dataset had **no missing values**.

```
mode_loan_amount_term = df['Loan_Amount_Term'].mode()[0]
mode_credit_history = df['Credit_History'].mode()[0]
```

9. Feature Engineering

We created a new feature **TotalIncome** by summing ApplicantIncome and CoapplicantIncome:

```
df['TotalIncome'] = df['ApplicantIncome'] + df['CoapplicantIncome']
df = df.drop(['ApplicantIncome', 'CoapplicantIncome'], axis=1)
```

This captures the combined earning capacity of applicants and co-applicants.

10. Outlier Detection and Treatment

Step 1: IQR Method

We identified outliers in LoanAmount and TotalIncome using the Interquartile Range (IQR) method.

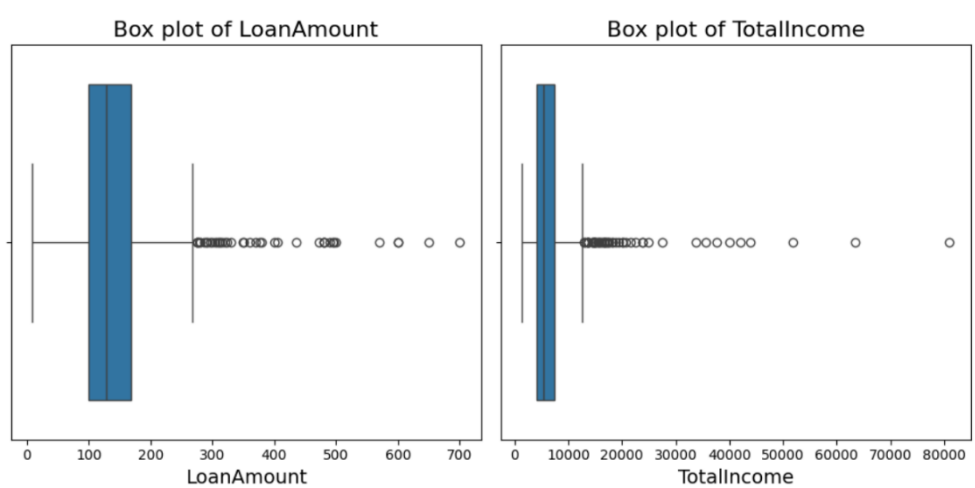
```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
```

Step 2: Visualization

Box plots were used to visualize outliers before and after treatment.



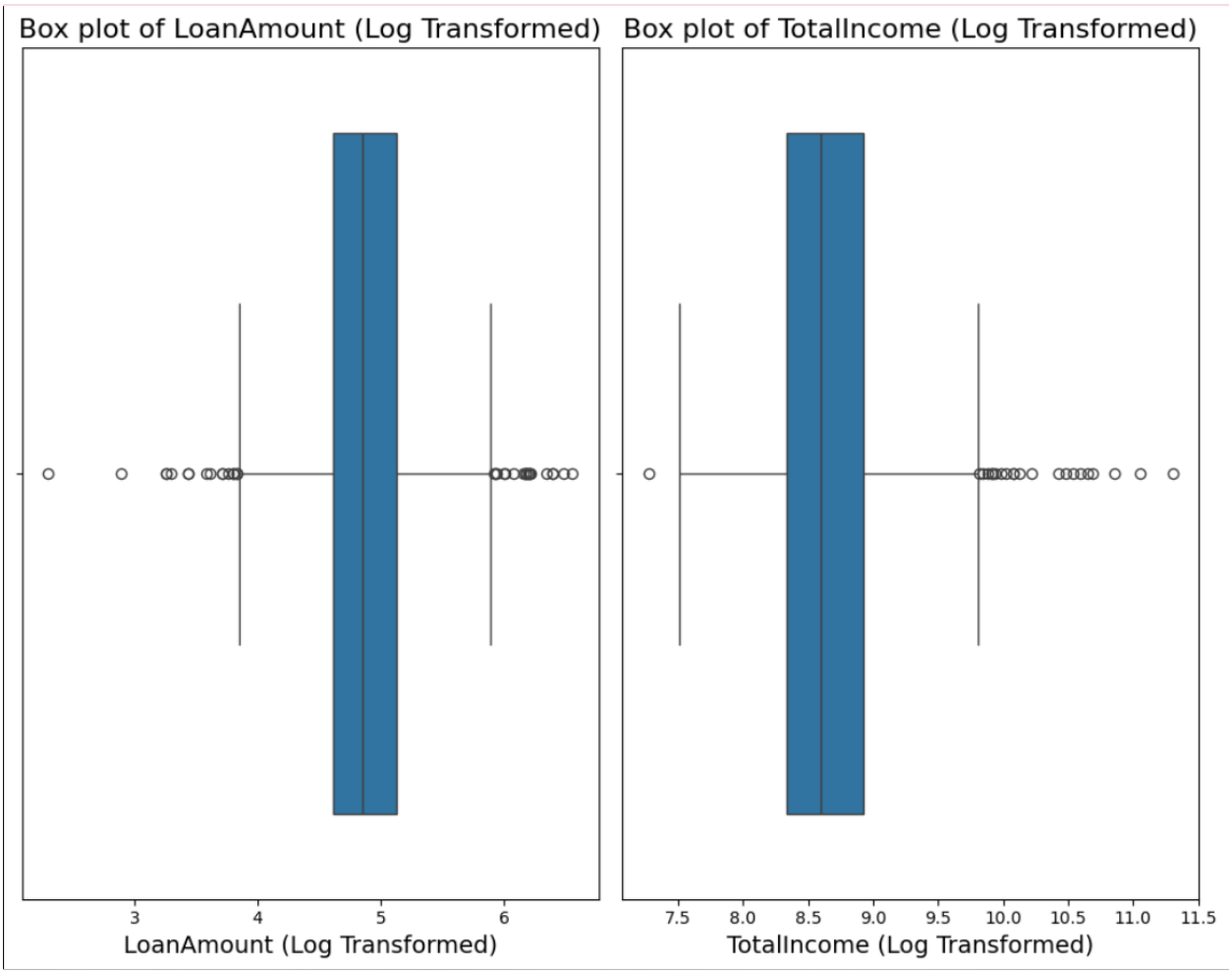
Box Plots of LoanAmount and TotalIncome.

Step 3: Log Transformation

Logarithmic transformation was applied to both columns:

```
df['LoanAmount_log'] = np.log(df['LoanAmount'] + 1)
df['TotalIncome_log'] = np.log(df['TotalIncome'] + 1)
```

This reduced skewness and improved distribution symmetry.



Box Plots of LoanAmount (Log Transformed) and TotalIncome (Log Transformed).

Step 4: Gentle Capping

Values above the 99.5th percentile were capped to prevent extreme outliers from distorting model performance.

```
for col in cols_to_cap:
    # Calculate the 0.995 quantile (99.5th percentile)
    cap_value = df[col].quantile(0.995)

    # Apply gentle capping using np.clip
    df[col] = np.clip(df[col], a_min=None, a_max=cap_value)
```

11. Encoding Categorical Variables

We categorized columns as **ordinal** and **nominal**.

- **Ordinal:** Dependents, Education
- **Nominal:** Gender, Married, Self_Employed, Property_Area

```
ordinal_cols = ['Dependents', 'Education']
nominal_cols = [col for col in df.select_dtypes(include='object').columns if col not in ordinal_cols + ['Loan_Status']]
```

Encoding Steps:

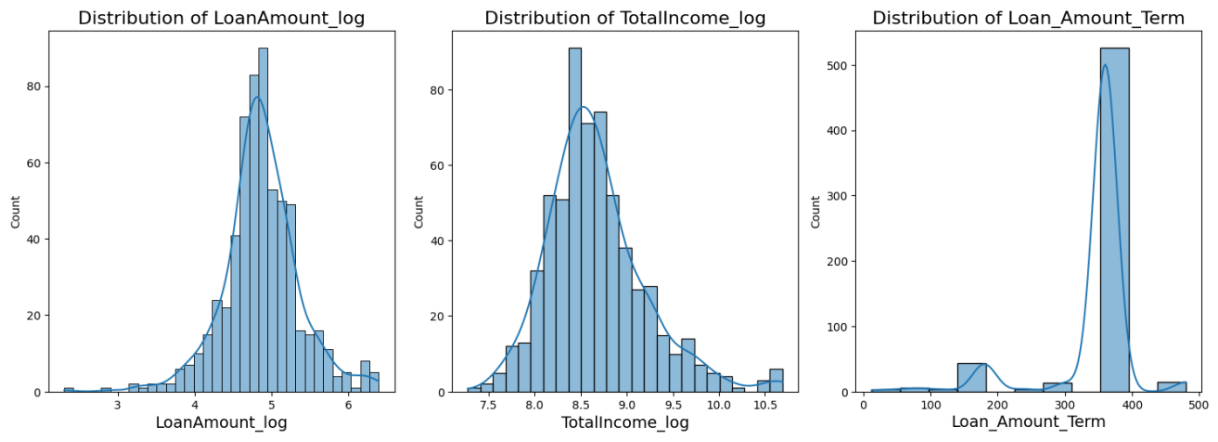
- Label Encoding for ordinal variables.
- One-Hot Encoding for nominal variables (with drop_first=True to prevent multicollinearity).
- Target variable (Loan_Status) was Label Encoded (Y=1, N=0).

12. Feature Scaling

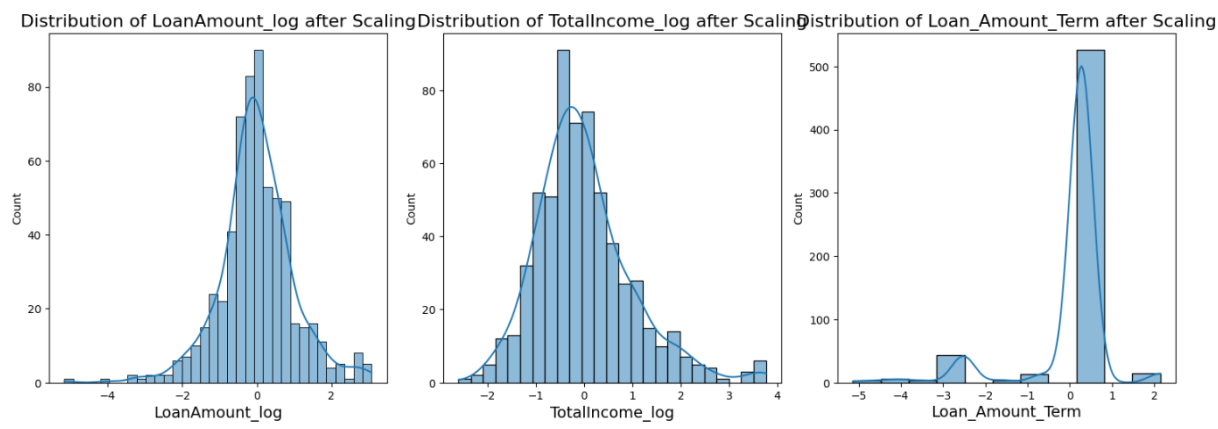
To normalize feature magnitudes, we applied **StandardScaler** to:

['LoanAmount_log', 'TotalIncome_log', 'Loan_Amount_Term']

Distributions were plotted before and after scaling to confirm normalization.



Distribution of numerical features (LoanAmount_log, TotalIncome_log, Loan_Amount_Term) before scaling. The plots illustrate the initial data distribution and highlight skewness or variability in the raw/log-transformed values.



Distribution of numerical features (LoanAmount_log, TotalIncome_log, Loan_Amount_Term) after scaling. The scaled plots demonstrate the effect of normalization/standardization, where features are now aligned to similar ranges and centered distributions, ensuring fair contribution to the model.

13. Summary of Preprocessing Steps

Step	Description
1	Loaded and inspected dataset
2	Dropped irrelevant column (Loan_ID)
3	Handled missing values logically and statistically
4	Predicted missing LoanAmount using RandomForest regression
5	Created new feature (TotalIncome)
6	Detected and treated outliers using IQR, log transform, and capping
7	Encoded categorical and target variables
8	Scaled numerical columns for model readiness